

Lab Session 10: The Web Developer (Async & APIs)

Total Estimated Time: 4 - 5 hours **Objective:** Master Asynchronous JavaScript. Learn to save user data locally so it survives a page refresh, fetch live data from external servers, and parse JSON to build a fully functional, real-world web application.

Part 1: The Web Storage API (Estimated Time: 1 hour)

Let's learn how to make data survive a page reload.

Activity 1: The Persistent Notepad

1. Create `notepad.html`.
2. Paste this starter code:

```
<!DOCTYPE html>
<html>
<body>
  <h2>My Secret Notes</h2>
  <textarea id="noteInput" rows="5" cols="30" placeholder="Type your secrets here...">
</textarea>
  <br>
  <button id="saveBtn">Save Note</button>
  <button id="clearBtn">Delete Saved Note</button>
  <p id="statusMessage"></p>

  <script>
    const noteBox = document.getElementById('noteInput');
    const saveBtn = document.getElementById('saveBtn');
    const clearBtn = document.getElementById('clearBtn');
    const statusMsg = document.getElementById('statusMessage');

    // --- 1. CHECK FOR SAVED DATA ON LOAD ---
    // Look inside localStorage for a key called 'mySecretNote'
    const savedNote = localStorage.getItem('mySecretNote');

    // If data exists, put it back inside the textarea!
    if (savedNote) {
      noteBox.value = savedNote;
      statusMsg.textContent = "Loaded previous note.";
    }

    // --- 2. SAVE DATA EVENT ---
    saveBtn.addEventListener('click', () => {
      // Get the text from the box
      const textToSave = noteBox.value;

      // Save it to localStorage
      localStorage.setItem('mySecretNote', textToSave);
      statusMsg.textContent = "Note saved to browser!";
    });
```

```

// --- 3. CLEAR DATA EVENT ---
clearBtn.addEventListener('click', () => {
  // Remove the specific key from localStorage
  localStorage.removeItem('mySecretNote');

  // Clear the text box on the screen
  noteBox.value = "";
  statusMsg.textContent = "Note permanently deleted.";
});
</script>
</body>
</html>

```

3. **Test it:** Open the page. Type a message. Click "Save". **Refresh the page.** Your text should still be there! Close the browser entirely and reopen the file. It survives!

Part 2: Fetching Data from the Internet (Estimated Time: 1.5 hours)

Let's use a free, public API that doesn't require an account to practice `fetch`.

Activity 2: The Random Joke Machine

We will use the Official Joke API (https://official-joke-api.appspot.com/random_joke).

1. Create `jokes.html`.
2. Paste this code:

```

<!DOCTYPE html>
<html>
<body>
  <h2>Random Joke Generator</h2>
  <button id="jokeBtn">Tell me a joke!</button>
  <h3 id="setupText">--</h3>
  <h3 id="punchlineText" style="color: blue;">--</h3>

  <script>
    const btn = document.getElementById('jokeBtn');
    const setupDisplay = document.getElementById('setupText');
    const punchlineDisplay = document.getElementById('punchlineText');

    // Notice the 'async' keyword! This allows us to use 'await' inside.
    btn.addEventListener('click', async () => {

      setupDisplay.textContent = "Thinking...";
      punchlineDisplay.textContent = "";

      // --- YOUR LOGIC HERE ---
      // 1. Fetch the data from the API URL.
      //    Example: const response = await fetch("URL_GOES_HERE");

      // 2. Convert the response to JSON.
      //    Example: const data = await response.json();
    });
  </script>

```

```

        // 3. Look at the data object. (Add console.log(data) to see it!)
        //     It has a 'setup' property and a 'punchline' property.
        //     Update setupDisplay.textContent with the setup.
        //     Update punchlineDisplay.textContent with the punchline.
    });
</script>
</body>
</html>

```

Part 3: The Grand Finale - Capstone Project (Estimated Time: 2 - 2.5 hours)

Activity 3: The Live Weather Dashboard

This project requires you to get a free API key. If you haven't yet, go to openweathermap.org, sign up, and get your API Key.

1. Create `weather.html`.
2. Paste this structure:

```

<!DOCTYPE html>
<html>
<style>
    body { font-family: Arial; display: flex; justify-content: center; margin-top: 50px;
background: #87CEEB;}
    .card { background: white; padding: 20px; border-radius: 10px; box-shadow: 0 4px 8px
rgba(0,0,0,0.2); text-align: center; width: 300px;}
    input { padding: 8px; width: 60%; }
    button { padding: 8px; background: #333; color: white; border: none; cursor: pointer; }
</style>
<body>
    <div class="card">
        <h2>Live Weather</h2>
        <input type="text" id="cityInput" placeholder="Enter city name">
        <button id="searchBtn">Search</button>

        <div id="weatherDisplay">
            <h1 id="cityName">--</h1>
            <h2 id="temperature">-- °C</h2>
            <p id="description">--</p>
        </div>
    </div>

    <script>
        // --- 1. SETUP ---
        const apiKey = "REPLACE_THIS_WITH_YOUR_ACTUAL_API_KEY"; // Keep the quotes!
        const searchBtn = document.getElementById('searchBtn');
        const cityInput = document.getElementById('cityInput');

        // --- 2. THE ASYNC FUNCTION ---
        async function fetchWeather(city) {

```

```

    try {
      // Construct the URL dynamically
      const url = `https://api.openweathermap.org/data/2.5/weather?
q=${city}&appid=${apiKey}&units=metric`;

      // Fetch the data
      const response = await fetch(url);

      // Error handling (e.g., if user types a city that doesn't exist)
      if (!response.ok) {
        alert("City not found!");
        return;
      }

      // Parse the JSON
      const data = await response.json();

      // Update the DOM
      document.getElementById('cityName').textContent = data.name;
      document.getElementById('temperature').textContent =
Math.round(data.main.temp) + " °C";
      document.getElementById('description').textContent =
data.weather[0].description.toUpperCase();

      // Save to localStorage
      localStorage.setItem('lastSearchedCity', city);

    } catch (error) {
      console.log("Something went wrong:", error);
    }
  }

// --- 3. EVENT LISTENER ---
searchBtn.addEventListener('click', () => {
  const requestedCity = cityInput.value;
  if(requestedCity !== "") {
    fetchWeather(requestedCity);
  }
});

// --- 4. ON PAGE LOAD ---
// Write logic here to check localStorage for 'lastSearchedCity'.
// If it exists, call fetchWeather() with that saved city so it loads automatically!
</script>
</body>
</html>

```

Part 4: The Final 20-Question Master Quiz (Estimated Time: 1 hour)

Section A: Synchronous vs. Asynchronous

1. By default, is JavaScript Synchronous or Asynchronous?

2. What happens to the webpage if a Synchronous task takes 5 seconds to complete?
3. What is the name of the JavaScript object that acts like a "receipt" for data that will arrive in the future?
4. A Promise has three states. Pending, _____, and Rejected.
5. What does API stand for?

Section B: Async/Await & Fetch 6. What keyword must be placed in front of a function definition to allow asynchronous behavior inside it? 7. What keyword is used to pause the execution of an async function until a Promise finishes? 8. What built-in browser function is used to make network requests to an API? 9. `fetch()` returns a: a) String b) Array c) Promise 10. If `response` is the result of a fetch request, what code do you write to convert that raw data into a usable JavaScript object?

Section C: JSON & Data 11. What does JSON stand for? 12. What does JSON data look almost exactly like? 13. True or False: You can only fetch data from an API if you have a paid account. 14. True or False: If a `fetch()` request fails (e.g., no internet), the entire JavaScript file will crash unless you handle the error.

Section D: Web Storage (`localStorage`) 15. What method is used to save data into `localStorage` ? 16. What method is used to read data from `localStorage` ? 17. What data type is the *only* data type that `localStorage` can save? a) Numbers b) Booleans c) Strings d) Objects 18. If a user closes their browser and restarts their computer, what happens to the data in `localStorage` ?

Section E: What's the Output & Find the Bug

19. What is the output order?

```
console.log("A");

async function getData() {
  await fetch("https://api.example.com");
  console.log("B");
}

getData();
console.log("C");
```

Output Order: __, __, __

20. Find the Bug:

```
async function getUser() {
  const response = fetch("https://api.example.com/user");
  const data = await response.json();
  console.log(data);
}
```

Bug: _____
